

SMT分拣机ECS接口调用说明书

文档概述

本文档基于SMT分拣机系统的JSON文件和C#数据模型定义，详细说明了所有可用的API接口端点、请求参数、返回数据结构及JSON示例字段说明。开发者可根据本文档准确理解如何根据模型定义构造请求及处理响应数据。

目录

- 1. [接口概览](#)
- 2. [数据模型定义](#)
- 3. [命令接口](#)
- 4. [查询接口](#)
- 5. [回调接口](#)
- 6. [错误处理](#)
- 7. [完整示例代码](#)

1. 接口概览

1.1 接口分类

接口类型	方向	说明
命令接口	WES → 设备	发送搬运、取消等命令
查询接口	WES → 设备	查询设备状态
回调接口	设备 → WES	任务结果回传、
事件推送接口	设备 → WES	设备事件推送 设备状态变化如扫码完成

1.2 接口列表

序号	接口名称	HTTP方法	端点	说明
1	发送搬运命令	POST	/api/v1/command	发送PICK_AND_PUT命令
2	取消命令	POST	api/v1/device/cancel	取消正在执行的命令
3	查询设备状态	GET	/api/v1/status	查询所有设备状态
4	任务结果回传	POST	[WES回调地址]	设备向WES回传任务结果
5	事件推送	POST	[WES事件上报地址]	设备向WES推送事件

1.3 通信协议

- 协议: HTTP/1.1
- 数据格式: JSON
- 字符编码: UTF-8
- Content-Type: application/json

2. 数据模型定义

基于 Smt.Models.cs 文件，定义以下数据模型：

2.1 PickAndPutCommand（搬运命令）

完整模型定义：

```
public class PickAndPutCommand
{
    [JsonPropertyName("device_id")]
    public string DeviceId { get; set; }           // 设备ID: ARM01/ARM02

    [JsonPropertyName("command_id")]
    public string CommandId { get; set; }         // 命令ID, 需全局唯一

    [JsonPropertyName("timestamp")]
    public string Timestamp { get; set; }         // 时间戳, 毫秒级

    [JsonPropertyName("task_type")]
    public string TaskType { get; set; }          // 命令类型: PICK_AND_PUT/SCAN/MOVE_FORWARD/MO

    [JsonPropertyName("priority")]
    public string Priority { get; set; }           // 优先级 (1-10, 10最高)

    [JsonPropertyName("timeout")]
    public string Timeout { get; set; }           // 期望完成时间(ms)

    [JsonPropertyName("source")]
    public LocationInfo Source { get; set; }       // 源库位信息

    [JsonPropertyName("target")]
    public LocationInfo Target { get; set; }       // 目标库位信息
}
```

2.2 LocationInfo (位置信息)

```
public class LocationInfo
{
    [JsonPropertyName("location_id")]
    public string LocationId { get; set; }           // 位置ID: STATION_A/STATION_B/STATION_SCAN/ST

    [JsonPropertyName("location_type")]
    public string LocationType { get; set; }         // 位置类型: ONE_LAYER_RACK/FIVE_LAYER_RACK/SC

    // ===== 料箱信息 =====
    [JsonPropertyName("bin_id")]
    public string BinId { get; set; }                // 箱子ID, ECS追溯用, 可选

    [JsonPropertyName("bin_type")]
    public string BinType { get; set; }              // 箱子类型: 三格箱/六格箱

    [JsonPropertyName("bin_location")]
    public string BinLocation { get; set; }          // 箱子位置: A/B/C/D, 单层货架上的储位

    [JsonPropertyName("bin_cell_location")]
    public string BinCellLocation { get; set; }      // 箱子单元格位置: 1-7

    // ===== 料盘信息 =====
    [JsonPropertyName("reel_layer")]
    public string ReelLayer { get; set; }            // 料盘层数

    [JsonPropertyName("reel_thickness")]
    public string ReelThickness { get; set; }        // 料盘厚度(mm)

    [JsonPropertyName("reel_totalthickness")]
    public string ReelTotalThickness { get; set; }   // 料盘总厚度(mm)

    [JsonPropertyName("reel_diameter")]
    public string ReelDiameter { get; set; }         // 料盘直径: 7寸/13寸/15寸

    // ===== 料架信息 =====
    [JsonPropertyName("rack_id")]
    public string RackId { get; set; }               // 料架ID, ECS追溯用, 可选

    [JsonPropertyName("rack_side")]
    public string RackSide { get; set; }             // 料架面数: A/B
}
```

```

[JsonPropertyName("rack_layer")]
public string RackLayer { get; set; } // 料架层数：1-5

[JsonPropertyName("rack_column")]
public string RackColumn { get; set; } // 料架列数
}

```

2.3 ApiResult (API响应)

```

public class ApiResult
{
    [JsonPropertyName("status")]
    public string Status { get; set; } // 状态：SUCCESS 或 FAILED

    [JsonPropertyName("message")]
    public string Message { get; set; } // 消息：成功或错误描述

    [JsonPropertyName("code")]
    public string Code { get; set; } // 状态码：200成功，400参数错误，503设备忙/故障

    [JsonPropertyName("trace_id")]
    public string TraceId { get; set; } // 可选，供应商内部日志ID-附带时间戳或command_id
}

```

3. 命令接口

3.1 发送搬运命令

接口描述: 向指定的机械臂发送搬运命令，控制机械臂从源位置取料并放到目标位置。

3.1.1 请求说明

HTTP方法: POST

请求端点: `http://<DEVICE_IP>:<PORT>/api/v1/device/command`

请求头:

3.1.2 请求参数

参数名	类型	必填	说明	示例值
device_id	string	是	设备ID，指定执行命令的机械臂	"ARM01" 或 "ARM02"
command_id	string	是	命令ID，需全局唯一	"CMD-20251215-1001"
timestamp	string	是	时间戳，毫秒级	"1702627300000"
task_type	string	是	任务类型	"PICK_AND_PUT"
priority	number	是	优先级(1-10)，10最高	1
timeout	number	是	期望完成时间(ms)	30000
source	object	是	源库位信息，见LocationInfo	-
target	object	是	目标库位信息，见LocationInfo	-

3.1.3 task_type 枚举值

值	说明
PICK_AND_PUT	搬运操作
SCAN	扫码操作
MOVE_FORWARD	前进
MOVE_BACKWARD	后退
MOVE_LEFT	左移
MOVE_RIGHT	右移

3.1.4 location_type 枚举值

值	说明	可用位置ID
ONE_LAYER_RACK	单层货架	STATION_A, STATION_B
FIVE_LAYER_RACK	五层货架	STATION_A, STATION_B

值	说明	可用位置ID
SCAN_PLATFORM	扫码平台	STATION_SCAN
SCAN_NG	NG平台	STATION_NG
BIN	流水线料箱	STATION_WORK1/STATTION_SCAN1

3.1.5 请求示例

场景1: 入库步骤1 - 从单层货架取料放到扫码平台

```
{
  "device_id": "ARM02",
  "command_id": "CMD-20251215-1001",
  "timestamp": "1702627300000",
  "task_type": "PICK_AND_PUT",
  "priority": 1,
  "timeout": 30000,
  "source": {
    "location_id": "STATION_A",
    "location_type": "ONE_LAYER_RACK",
    "rack_id": "RACK_001",
    "bin_id": "BIN_104",
    "bin_type": "三格箱",
    "bin_location": "A",
    "bin_cell_location": "1",
    "reel_layer": "10",
    "reel_thickness": "20",
    "reel_diameter": "15",
    "reel_totalthickness": "200"
  },
  "target": {
    "location_id": "STATION_SCAN",
    "location_type": "SCAN_PLATFORM"
  }
}
```

字段说明:

- device_id : "ARM02" - 使用2号机械臂执行
- source : 单层货架位置，包含完整的料箱和料盘信息
 - location_id : "STATION_A" - A侧单层货架

- location_type: "ONE_LAYER_RACK" - 单层货架类型
- bin_location: "A" - 料箱在货架上的A位置
- bin_cell_location: "1" - 料箱的第1个单元格
- reel_layer: "10" - 料盘第10层
- reel_thickness: "20" - 料盘厚度20mm
- reel_diameter: "15" - 15寸料盘
- reel_totalthickness: "200" - 料盘总厚度200mm
- target: 扫码平台, 仅需location_id和location_type

场景2: 入库步骤2 - 从扫码平台取料放到流水线料箱

```
{
  "device_id": "ARM01",
  "command_id": "CMD-20251215-1002",
  "timestamp": "1702627300000",
  "task_type": "PICK_AND_PUT",
  "priority": 1,
  "timeout": 30000,
  "source": {
    "location_id": "STATION_SCAN",
    "location_type": "SCAN_PLATFORM"
  },
  "target": {
    "location_id": "STATION_WORK1",
    "location_type": "BIN",
    "bin_type": "三格箱",
    "bin_cell_location": "1",
    "reel_layer": "15",
    "reel_thickness": "20",
    "reel_diameter": "15",
    "reel_totalthickness": "300"
  }
}
```

字段说明:

- device_id: "ARM01" - 使用1号机械臂执行
- source: 扫码平台, 仅需location_id和location_type
- target: 流水线料箱位置

- location_id : "STATION_WORK1" - 流水线工作站1
- location_type : "BIN" - 料箱类型
- bin_type : "三格箱" - 三格箱类型
- bin_cell_location : "1" - 第1个单元格
- reel_layer : "15" - 料盘第15层 (堆叠层数)
- reel_thickness : "20" - 料盘厚度20mm
- reel_totalthickness : "300" - 料盘总厚度300mm

场景3: 出库步骤1 - 从流水线料箱取料放到扫码平台

```
{  
  "device_id": "ARM01",  
  "command_id": "CMD-20251215-1001",  
  "timestamp": "1702627300000",  
  "task_type": "PICK_AND_PUT",  
  "priority": 1,  
  "timeout": 30000,  
  "source": {  
    "location_id": "STATION_WORK1",  
    "location_type": "BIN",  
    "bin_type": "三格箱",  
    "bin_cell_location": "1",  
    "reel_layer": "10",  
    "reel_thickness": "20",  
    "reel_diameter": "15",  
    "reel_totalthickness": "200"  
  },  
  "target": {  
    "location_id": "STATION_SCAN",  
    "location_type": "SCAN_PLATFORM"  
  }  
}
```

字段说明:

- device_id : "ARM01" - 使用1号机械臂执行
- source : 流水线料箱位置, 包含料箱和料盘信息
- target : 扫码平台, 仅需location_id和location_type

场景4: 出库步骤2 - 从扫码平台取料放到五层货架

```
{
  "device_id": "ARM02",
  "command_id": "CMD-20251215-1001",
  "timestamp": "1702627300000",
  "task_type": "PICK_AND_PUT",
  "priority": 1,
  "timeout": 30000,
  "source": {
    "location_id": "STATION_SCAN",
    "location_type": "SCAN_PLATFORM"
  },
  "target": {
    "location_id": "STATION_A",
    "location_type": "FIVE_LAYER_RACK",
    "rack_side": "A",
    "rack_layer": "1",
    "rack_column": "1",
    "reel_thickness": "20",
    "reel_diameter": "15"
  }
}
```

字段说明:

- device_id : "ARM02" - 使用2号机械臂执行
- source : 扫码平台, 仅需location_id和location_type
- target : 五层货架位置
 - location_id : "STATION_A" - A侧五层货架
 - location_type : "FIVE_LAYER_RACK" - 五层货架类型
 - rack_side : "A" - A面
 - rack_layer : "1" - 第1层
 - rack_column : "1" - 第1列
 - reel_thickness : "20" - 料盘厚度20mm
 - reel_diameter : "15" - 15寸料盘

场景5: 移库步骤1 - 从五层货架取料放到扫码平台

```
{
  "device_id": "ARM02",
  "command_id": "CMD-20251215-1001",
  "timestamp": "1702627300000",
  "task_type": "PICK_AND_PUT",
  "priority": 1,
  "timeout": 30000,
  "source": {
    "location_id": "STATION_A",
    "location_type": "FIVE_LAYER_RACK",
    "rack_id": "RACK_01",
    "rack_side": "A",
    "rack_layer": "1",
    "rack_column": "1",
    "reel_thickness": "20",
    "reel_diameter": "15"
  },
  "target": {
    "location_id": "STATION_SCAN",
    "location_type": "SCAN_PLATFORM"
  }
}
```

字段说明:

- device_id : "ARM02" - 使用2号机械臂执行
- source : 五层货架位置
 - rack_id : "RACK_01" - 货架ID
 - rack_side : "A" - A面
 - rack_layer : "1" - 第1层
 - rack_column : "1" - 第1列
- target : 扫码平台

场景6: 移库步骤2 - 从扫码平台取料放到五层货架

```
{
  "device_id": "ARM02",
  "command_id": "CMD-20251215-1001",
  "timestamp": "1702627300000",
  "task_type": "PICK_AND_PUT",
  "priority": 1,
  "timeout": 30000,
  "source": {
    "location_id": "STATION_SCAN",
    "location_type": "SCAN_PLATFORM"
  },
  "target": {
    "location_id": "STATION_B",
    "location_type": "FIVE_LAYER_RACK",
    "rack_id": "RACK_02",
    "rack_side": "A",
    "rack_layer": "1",
    "rack_column": "1",
    "reel_thickness": "20",
    "reel_diameter": "15"
  }
}
```

字段说明:

- device_id : "ARM02" - 使用2号机械臂执行
- source : 扫码平台
- target : 五层货架位置
 - location_id : "STATION_B" - B侧五层货架
 - rack_id : "RACK_02" - 目标货架ID

3.1.6 响应说明

响应结构: ApiResult

字段名	类型	说明
status	string	状态: "SUCCESS" 或 "FAILED"
message	string	消息: "Accepted" 或错误描述

字段名	类型	说明
code	string	状态码: 200成功接收, 400参数错误, 503设备忙/故障
trace_id	string	可选, 供应商内部日志ID-附带时间戳或command_id

成功响应示例:

```
{
  "status": "SUCCESS",
  "message": "Accepted",
  "code": "200",
  "trace_id": "DEV-LOG-1702627300000"
}
```

失败响应示例:

```
{
  "status": "FAILED",
  "message": "Parameter error: device_id is required",
  "code": "400",
  "trace_id": "DEV-LOG-CMD-20251215-1001"
}
```

3.2 取消命令

接口描述: 取消正在执行的命令。

3.2.1 请求说明

HTTP方法: POST

请求端点: /api/v1/command/cancel

3.2.2 请求参数

参数名	类型	必填	说明	示例值
command_id	string	是	要取消的命令ID	"CMD-20251215-1001"

3.2.3 请求示例

```
{  
  "command_id": "CMD-20251215-1001"  
}
```

3.2.4 响应说明

成功响应示例:

```
{  
  "status": "SUCCESS",  
  "message": "Cancelled",  
  "code": "200",  
  "trace_id": "DEV-LOG-CMD-20251215-1001"  
}
```

失败响应示例:

```
{  
  "status": "FAILED",  
  "message": "Command not found or already completed",  
  "code": "400",  
  "trace_id": "DEV-LOG-CMD-20251215-1001"  
}
```

4. 查询接口

4.1 查询设备状态

接口描述: 查询所有机械臂的当前状态。

4.1.1 请求说明

HTTP方法: GET

请求端点: `http://<DEVICE_IP>:<PORT>/api/v1/device/status`

4.1.2 请求参数

无请求参数。

4.1.3 响应说明

响应结构: DeviceStatusArray

字段名	类型	说明
status	array	设备状态数组，每个元素包含一个设备的状态信息

DeviceStatus对象:

字段名	类型	说明	示例值
device_id	string	设备ID: ARM01/ARM02	"ARM01"
timestamp	number	时间戳(毫秒级)	1702627300000
current_cmd_id	string	当前正在执行的命令ID，空闲时为空	"CMD-20251215-1001"
error_code	string	错误代码，无错误时为"NONE"	"NONE"
status	string	设备状态	"RUNNING"

设备状态枚举值:

值	说明
IDLE	空闲
RUNNING	运行中
ERROR	错误
OFFLINE	离线

4.1.4 响应示例

```
{
  "status": [
    {
      "device_id": "ARM01",
      "timestamp": 1702627300000,
      "current_cmd_id": "CMD-20251215-1001",
      "error_code": "NONE",
      "status": "RUNNING"
    },
    {
      "device_id": "ARM02",
      "timestamp": 1702627300000,
      "current_cmd_id": "CMD-20251215-1001",
      "error_code": "NONE",
      "status": "RUNNING"
    }
  ]
}
```

字段说明:

- 返回两个设备的状态数组
- device_id: "ARM01" 和 "ARM02", 两个机械臂
- current_cmd_id: 如果设备正在执行命令, 返回command_id; 如果空闲, 该字段为null或空字符串
- error_code: "NONE"表示无错误, 其他值表示具体错误类型
- status: "RUNNING"表示设备正在运行命令

5. 回调接口

回调接口由设备主动向WES（仓储执行系统）发起，用于回传任务执行结果或推送事件。

5.1 任务结果回传（Report Result）

接口描述: 设备完成搬运命令后，向WES回传任务执行结果。

5.1.1 请求说明

HTTP方法: POST

请求端点: http://<WES_IP>:<PORT>/api/v1/callback/result]

5.1.2 请求参数

参数名	类型	必填	说明	示例值
command_id	string	是	必须回传原指令ID	"CMD-20251215-1001"
device_id	string	是	设备ID	"ARM_01"
result	string	是	执行结果: SUCCESS 或 FAILED	"SUCCESS"
finish_time	number	是	完成时间戳(毫秒级)	1702627250000
data	object	否	业务回传数据	-
error_detail	object	否	错误详情(result=FAILED时必填)	-

data对象字段:

字段名	类型	说明	示例值
actual_qty	number	实际操作数量	1
pick_and_put_result	string	放料指令具体结果	"PUT_FINISHED"

pick_and_put_result枚举值:

值	说明
PUT_FINISHED	放料完成
PUT_FAILED	放料失败
PICK_FAILED	取料失败

error_detail对象字段:

字段名	类型	说明	示例值
error_code	string	错误码, 0为无错误	"0"

字段名	类型	说明	示例值
error_message	string	具体错误信息	""

5.1.3 请求示例

成功回传示例:

```
{
  "command_id": "CMD-20251215-1001",
  "device_id": "ARM_01",
  "result": "SUCCESS",
  "finish_time": 1702627250000,
  "data": {
    "actual_qty": 1,
    "pick_and_put_result": "PUT_FINISHED"
  }
}
```

失败回传示例:

```
{
  "command_id": "CMD-20251215-1001",
  "device_id": "ARM_01",
  "result": "FAILED",
  "finish_time": 1702627250000,
  "data": {
    "actual_qty": 0,
    "pick_and_put_result": "PUT_FAILED"
  },
  "error_detail": {
    "error_code": "1001",
    "error_message": "Target position blocked"
  }
}
```

字段说明:

- command_id : 必须与原发送的命令ID一致，用于任务追踪
- result : "SUCCESS"表示命令执行成功，"FAILED"表示执行失败
- finish_time : 任务完成的精确时间戳

- data.actual_qty : 实际完成的搬运数量
- data.pick_and_put_result : 具体执行结果, 成功时为"PUT_FINISHED"
- error_detail : 仅在result="FAILED"时需要提供, 说明失败原因

5.1.4 响应说明

WES接收到回调后, 需要返回确认响应。

成功响应示例:

```
{  
  "status": "SUCCESS",  
  "message": "ACK",  
  "code": "200",  
  "trace_id": "DEV-LOG-1702627300000"  
}
```

失败响应示例:

```
{  
  "status": "FAILED",  
  "message": "Invalid parameters",  
  "code": "400",  
  "trace_id": "DEV-LOG-1702627300000"  
}
```

5.2 事件推送 (Event Push)

接口描述: 扫码设备完成扫码后, 向WES推送扫码事件。

5.2.1 请求说明

HTTP方法: POST

请求端点: [http://<WES_IP>:<PORT>/api/v1/callback/event]

5.2.2 请求参数

参数名	类型	必填	说明	示例值
device_id	string	是	设备ID	"SCANNER_001"
event_type	string	是	事件类型	"SCAN_COMPLETED"
timestamp	number	是	事件时间戳(毫秒级)	1702627300000
data	object	否	业务负载数据	-

event_type枚举值:

值	说明
SCAN_COMPLETED	扫码完成
ESTOP_PRESSED	紧急停止

data对象字段:

字段名	类型	说明	示例值
location	string	扫码位置	"STATION_SCAN"
barcode	string	条码信息	"PKG12345678"

5.2.3 请求示例

扫码完成事件示例:

```
{
  "device_id": "SCANNER_001",
  "event_type": "SCAN_COMPLETED",
  "timestamp": 1702627300000,
  "data": {
    "location": "STATION_SCAN",
    "barcode": "PKG12345678"
  }
}
```

紧急停止事件示例:

```
{
  "device_id": "SCANNER_001",
  "event_type": "ESTOP_PRESSED",
  "timestamp": 1702627300000
}
```

字段说明:

- device_id : 扫码设备的ID
- event_type : "SCAN_COMPLETED"表示扫码完成, "ESTOP_PRESSED"表示紧急停止
- timestamp : 事件发生的时间戳
- data.location : 扫码位置, 通常为"STATION_SCAN"
- data.barcode : 扫码获取的条码信息

5.2.4 响应说明

WES接收到事件推送后, 需要返回确认响应。

成功响应示例:

```
{
  "status": "SUCCESS",
  "message": "ACK",
  "code": "200",
  "trace_id": "DEV-LOG-1702627300000"
}
```

6. 错误处理

6.1 HTTP状态码

状态码	说明	处理建议
200	成功	正常处理响应数据
400	请求参数错误	检查请求参数格式和必填字段
503	设备忙/故障	等待设备空闲后重试或联系技术支持

6.2 业务错误码

code	说明	处理建议
200	成功	-
400	参数错误	检查参数格式
503	设备忙	等待后重试

6.3 设备错误码 (error_code)

error_code	说明	处理建议
NONE	无错误	-
1001	目标位置被阻挡	清理障碍物后重试
1002	取料失败	检查物料位置是否正确
1003	放料失败	检查目标位置是否可访问

6.4 错误处理示例

```
public async Task<bool> SendCommandWithErrorHandling(PickAndPutCommand command, int maxRetries :
{
    int retryCount = 0;

    while (retryCount < maxRetries)
    {
        try
        {
            var result = await SendCommandAsync(command);

            if (result.Status == "SUCCESS" && result.Code == "200")
            {
                return true;
            }

            // 设备忙碌，等待后重试
            if (result.Code == "503")
            {
                Console.WriteLine($"设备忙碌，等待后重试... (第{retryCount + 1}次)");
                await Task.Delay(5000);
                retryCount++;
                continue;
            }

            // 参数错误，不重试
            if (result.Code == "400")
            {
                Console.WriteLine($"参数错误: {result.Message}");
                return false;
            }

            // 其他错误，不重试
            Console.WriteLine($"命令发送失败: {result.Message} (错误码: {result.Code})");
            return false;
        }
        catch (Exception ex)
        {
            retryCount++;
            Console.WriteLine($"发送命令异常: {ex.Message} (第{retryCount}次)");

            if (retryCount >= maxRetries)
```

```
{  
    Console.WriteLine("已达到最大重试次数，放弃发送");  
    return false;  
}  
  
    await Task.Delay(5000);  
}  
}  
  
return false;  
}
```

7. 完整示例代码

7.1 数据模型定义

```
using System.Text.Json.Serialization;

namespace Smt.Models
{
    /// <summary>
    /// 搬运命令
    /// </summary>
    public class PickAndPutCommand
    {
        [JsonPropertyName("device_id")]
        public string DeviceId { get; set; }

        [JsonPropertyName("command_id")]
        public string CommandId { get; set; }

        [JsonPropertyName("timestamp")]
        public string Timestamp { get; set; }

        [JsonPropertyName("task_type")]
        public string TaskType { get; set; }

        [JsonPropertyName("priority")]
        public string Priority { get; set; }

        [JsonPropertyName("timeout")]
        public string Timeout { get; set; }

        [JsonPropertyName("source")]
        public LocationInfo Source { get; set; }

        [JsonPropertyName("target")]
        public LocationInfo Target { get; set; }
    }

    /// <summary>
    /// 位置信息
    /// </summary>
    public class LocationInfo
```

```
{  
    [JsonPropertyName("location_id")]  
    public string LocationId { get; set; }  
  
    [JsonPropertyName("location_type")]  
    public string LocationType { get; set; }  
  
    [JsonPropertyName("bin_id")]  
    public string BinId { get; set; }  
  
    [JsonPropertyName("bin_type")]  
    public string BinType { get; set; }  
  
    [JsonPropertyName("bin_location")]  
    public string BinLocation { get; set; }  
  
    [JsonPropertyName("bin_cell_location")]  
    public string BinCellLocation { get; set; }  
  
    [JsonPropertyName("reel_layer")]  
    public string ReelLayer { get; set; }  
  
    [JsonPropertyName("reel_thickness")]  
    public string ReelThickness { get; set; }  
  
    [JsonPropertyName("reel_totalthickness")]  
    public string ReelTotalThickness { get; set; }  
  
    [JsonPropertyName("reel_diameter")]  
    public string ReelDiameter { get; set; }  
  
    [JsonPropertyName("rack_id")]  
    public string RackId { get; set; }  
  
    [JsonPropertyName("rack_side")]  
    public string RackSide { get; set; }  
  
    [JsonPropertyName("rack_layer")]  
    public string RackLayer { get; set; }  
  
    [JsonPropertyName("rack_column")]  
    public string RackColumn { get; set; }  
}
```

```
/// <summary>
/// API响应
/// </summary>
public class ApiResult
{
    [JsonPropertyName("status")]
    public string Status { get; set; }

    [JsonPropertyName("message")]
    public string Message { get; set; }

    [JsonPropertyName("code")]
    public string Code { get; set; }

    [JsonPropertyName("trace_id")]
    public string TraceId { get; set; }
}

/// <summary>
/// 设备状态
/// </summary>
public class DeviceStatus
{
    [JsonPropertyName("device_id")]
    public string DeviceId { get; set; }

    [JsonPropertyName("timestamp")]
    public long Timestamp { get; set; }

    [JsonPropertyName("current_cmd_id")]
    public string CurrentCmdId { get; set; }

    [JsonPropertyName("error_code")]
    public string ErrorCode { get; set; }

    [JsonPropertyName("status")]
    public string Status { get; set; }
}

/// <summary>
/// 设备状态数组
/// </summary>
```

```
public class DeviceStatusArray
{
    [JsonPropertyName("status")]
    public List<DeviceStatus> Status { get; set; }
}
```

```
/// <summary>
/// 取消命令请求
/// </summary>
```

```
public class CancelCommandRequest
{
    [JsonPropertyName("command_id")]
    public string CommandId { get; set; }
}
```

```
/// <summary>
/// 任务结果报告
/// </summary>
```

```
public class TaskResultReport
{
    [JsonPropertyName("command_id")]
    public string CommandId { get; set; }

    [JsonPropertyName("device_id")]
    public string DeviceId { get; set; }

    [JsonPropertyName("result")]
    public string Result { get; set; }

    [JsonPropertyName("finish_time")]
    public long FinishTime { get; set; }

    [JsonPropertyName("data")]
    public TaskResultData Data { get; set; }

    [JsonPropertyName("error_detail")]
    public ErrorDetail ErrorDetail { get; set; }
}
```

```
/// <summary>
/// 任务结果数据
/// </summary>
```

```
public class TaskResultData
```

```
{
    [JsonPropertyName("actual_qty")]
    public int ActualQty { get; set; }

    [JsonPropertyName("pick_and_put_result")]
    public string PickAndPutResult { get; set; }
}
```

```
/// <summary>
/// 错误详情
/// </summary>
public class ErrorDetail
{
    [JsonPropertyName("error_code")]
    public string ErrorCode { get; set; }

    [JsonPropertyName("error_message")]
    public string ErrorMessage { get; set; }
}
```

```
/// <summary>
/// 事件推送
/// </summary>
public class EventPush
{
    [JsonPropertyName("device_id")]
    public string DeviceId { get; set; }

    [JsonPropertyName("event_type")]
    public string EventType { get; set; }

    [JsonPropertyName("timestamp")]
    public long Timestamp { get; set; }

    [JsonPropertyName("data")]
    public EventData Data { get; set; }
}
```

```
/// <summary>
/// 事件数据
/// </summary>
public class EventData
{

```

```
[JsonPropertyName("location")]  
public string Location { get; set; }
```

```
[JsonPropertyName("barcode")]  
public string Barcode { get; set; }
```

```
}
```

```
}
```

7.2 API客户端实现

```
using System;
using System.Net.Http;
using System.Text;
using System.Text.Json;
using System.Threading.Tasks;
using Smt.Models;

namespace Smt.Client
{
    public class SmtSorterApiClient
    {
        private readonly HttpClient _httpClient;
        private readonly string _baseUrl;

        public SmtSorterApiClient(string baseUrl)
        {
            _baseUrl = baseUrl;
            _httpClient = new HttpClient
            {
                BaseAddress = new Uri(baseUrl)
            };
            _httpClient.DefaultRequestHeaders.Accept.Add(
                new System.Net.Http.Headers.MediaTypeWithQualityHeaderValue("application/json"));
        }

        /// <summary>
        /// 发送搬运命令
        /// </summary>
        public async Task<ApiResult> SendPickAndPutCommandAsync(PickAndPutCommand command)
        {
            var json = JsonSerializer.Serialize(command);
            var content = new StringContent(json, Encoding.UTF8, "application/json");

            var response = await _httpClient.PostAsync("/api/v1/command", content);
            var responseContent = await response.Content.ReadAsStringAsync();

            return JsonSerializer.Deserialize<ApiResult>(responseContent);
        }

        /// <summary>
        /// 取消命令
    }
}
```

```

/// </summary>
public async Task<ApiResult> CancelCommandAsync(string commandId)
{
    var request = new CancelCommandRequest { CommandId = commandId };
    var json = JsonSerializer.Serialize(request);
    var content = new StringContent(json, Encoding.UTF8, "application/json");

    var response = await _httpClient.PostAsync("/api/v1/command/cancel", content);
    var responseContent = await response.Content.ReadAsStringAsync();

    return JsonSerializer.Deserialize<ApiResult>(responseContent);
}

/// <summary>
/// 查询设备状态
/// </summary>
public async Task<DeviceStatusArray> GetDeviceStatusAsync()
{
    var response = await _httpClient.GetAsync("/api/v1/status");
    var content = await response.Content.ReadAsStringAsync();

    return JsonSerializer.Deserialize<DeviceStatusArray>(content);
}

/// <summary>
/// 创建入库命令（步骤1：从货架到扫码平台）
/// </summary>
public PickAndPutCommand CreateInboundStep1Command(
    string deviceId, string rackId, string binId, string binType,
    string binLocation, string binCellLocation, string reelLayer,
    string reelThickness, string reelDiameter, string reelTotalThickness)
{
    return new PickAndPutCommand
    {
        DeviceId = deviceId,
        CommandId = $"CMD-{DateTime.Now:yyyyMMdd-HH:mm:ss}",
        Timestamp = DateTimeOffset.Now.ToUnixTimeMilliseconds().ToString(),
        TaskType = "PICK_AND_PUT",
        Priority = "1",
        Timeout = "30000",
        Source = new LocationInfo
        {
            LocationId = "STATION_A",

```

```

        LocationType = "ONE_LAYER_RACK",
        RackId = rackId,
        BinId = binId,
        BinType = binType,
        BinLocation = binLocation,
        BinCellLocation = binCellLocation,
        ReelLayer = reelLayer,
        ReelThickness = reelThickness,
        ReelDiameter = reelDiameter,
        ReelTotalThickness = reelTotalThickness
    },
    Target = new LocationInfo
    {
        LocationId = "STATION_SCAN",
        LocationType = "SCAN_PLATFORM"
    }
};
}

/// <summary>
/// 创建入库命令（步骤2：从扫码平台到料箱）
/// </summary>
public PickAndPutCommand CreateInboundStep2Command(
    string deviceId, string binType, string binCellLocation,
    string reelLayer, string reelThickness, string reelDiameter,
    string reelTotalThickness)
{
    return new PickAndPutCommand
    {
        DeviceId = deviceId,
        CommandId = $"CMD-{DateTime.Now:yyyyMMdd-HH:mm:ss}",
        Timestamp = DateTimeOffset.Now.ToUnixTimeMilliseconds().ToString(),
        TaskType = "PICK_AND_PUT",
        Priority = "1",
        Timeout = "30000",
        Source = new LocationInfo
        {
            LocationId = "STATION_SCAN",
            LocationType = "SCAN_PLATFORM"
        },
        Target = new LocationInfo
        {
            LocationId = "STATION_WORK1",

```

```
        LocationType = "BIN",
        BinType = binType,
        BinCellLocation = binCellLocation,
        ReelLayer = reelLayer,
        ReelThickness = reelThickness,
        ReelDiameter = reelDiameter,
        ReelTotalThickness = reelTotalThickness
    }
};
}
}
```

7.3 使用示例

```
using System;
using System.Threading.Tasks;
using Smt.Client;
using Smt.Models;

class Program
{
    static async Task Main(string[] args)
    {
        // 初始化API客户端
        var client = new SmtSorterApiClient("http://192.168.1.100:8080");

        // ===== 示例1: 查询设备状态 =====
        Console.WriteLine("查询设备状态...");
        var statusArray = await client.GetDeviceStatusAsync();
        foreach (var status in statusArray.Status)
        {
            Console.WriteLine($"设备 {status.DeviceId}: 状态={status.Status}, 当前命令={status.Ci
        }

        // ===== 示例2: 执行入库流程 =====
        Console.WriteLine("\n开始入库流程...");

        // 步骤1: 从货架到扫码平台
        var step1 = client.CreateInboundStep1Command(
            deviceId: "ARM02",
            rackId: "RACK_001",
            binId: "BIN_104",
            binType: "三格箱",
            binLocation: "A",
            binCellLocation: "1",
            reelLayer: "10",
            reelThickness: "20",
            reelDiameter: "15",
            reelTotalThickness: "200"
        );

        var result1 = await client.SendPickAndPutCommandAsync(step1);
        Console.WriteLine($"步骤1结果: {result1.Status} - {result1.Message} (Code: {result1.Code

        if (result1.Status == "SUCCESS")
```

```

{
    Console.WriteLine($"命令 {step1.CommandId} 发送成功");
}

// 等待扫码完成（模拟）
await Task.Delay(5000);

// 步骤2：从扫码平台到料箱
var step2 = client.CreateInboundStep2Command(
    deviceId: "ARM01",
    binType: "三格箱",
    binCellLocation: "1",
    reelLayer: "15",
    reelThickness: "20",
    reelDiameter: "15",
    reelTotalThickness: "300"
);

var result2 = await client.SendPickAndPutCommandAsync(step2);
Console.WriteLine($"步骤2结果: {result2.Status} - {result2.Message} (Code: {result2.Code}

// ===== 示例3：取消命令 =====
Console.WriteLine("\n取消命令示例...");
var cancelResult = await client.CancelCommandAsync(step2.CommandId);
Console.WriteLine($"取消结果: {cancelResult.Status} - {cancelResult.Message}");
}
}

```

8. 附录

8.1 位置类型必填字段速查表

location_type	必填字段	可选字段
ONE_LAYER_RACK	location_id, location_type, bin_type, bin_location, bin_cell_location, reel_layer, reel_thickness, reel_diameter, reel_totalthickness	rack_id, bin_id

location_type	必填字段	可选字段
FIVE_LAYER_RACK	location_id, location_type, rack_id, rack_side, rack_layer, rack_column, reel_thickness, reel_diameter	-
SCAN_PLATFORM	location_id, location_type	-
BIN	location_id, location_type, bin_type, bin_cell_location, reel_layer, reel_thickness, reel_diameter, reel_totalthickness	-

8.2 料盘直径规格

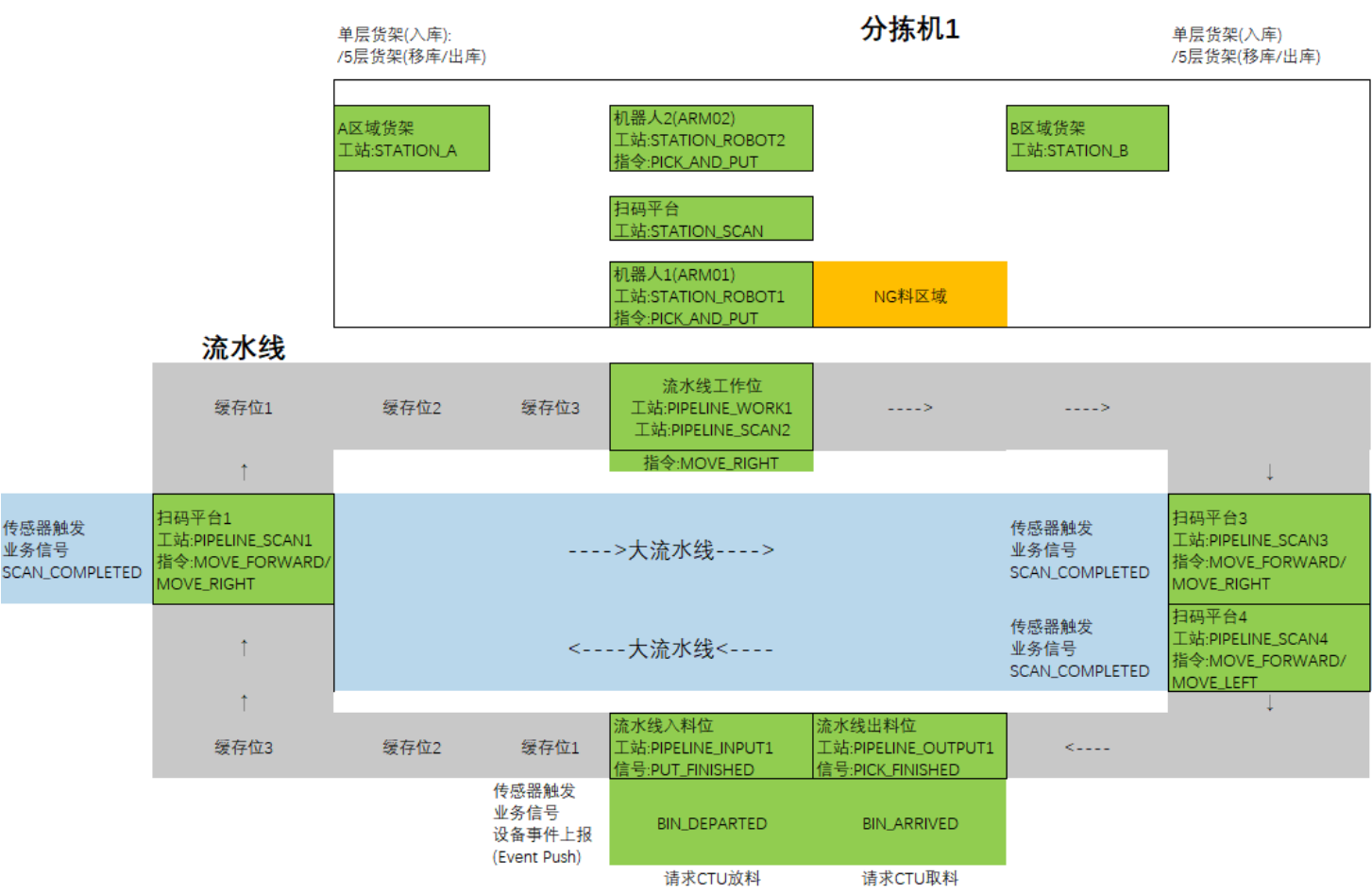
直径	说明	适用场景
7寸	小型料盘	小型元件
13寸	中型料盘	中型元件
15寸	大型料盘	大型元件

8.3 术语表

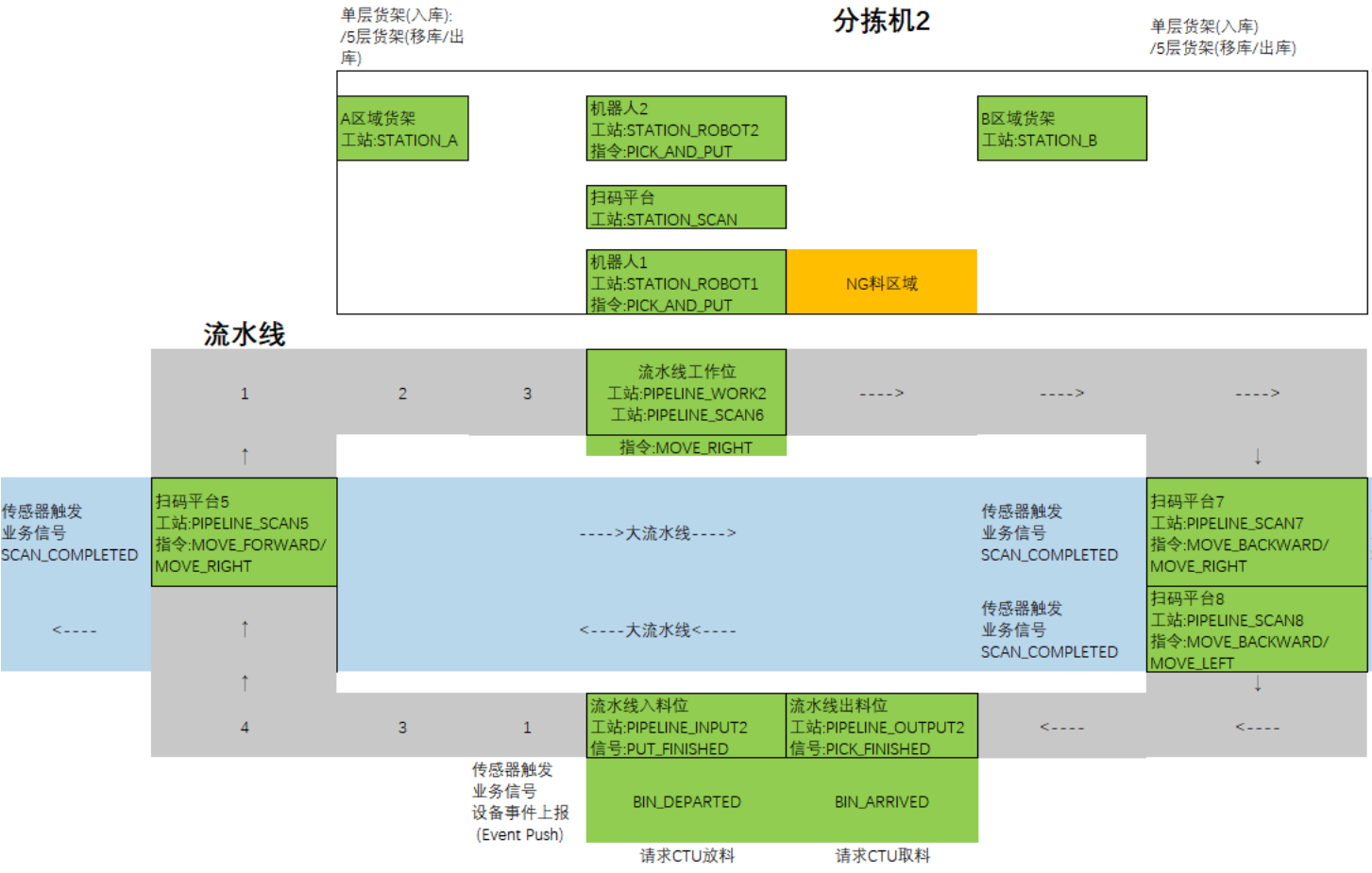
术语	英文	说明
WES	Warehouse Execution System	仓储执行系统
ECS	Enterprise Control System	企业控制系统
PICK_AND_PUT	拾取与放置	搬运操作
REEL	料盘	存放电子元件的盘状容器
BIN	料箱	存放料盘的容器
RACK	货架	存放料箱的架式设备
TRACE_ID	追踪ID	用于日志追踪的唯一标识

9. 布局图

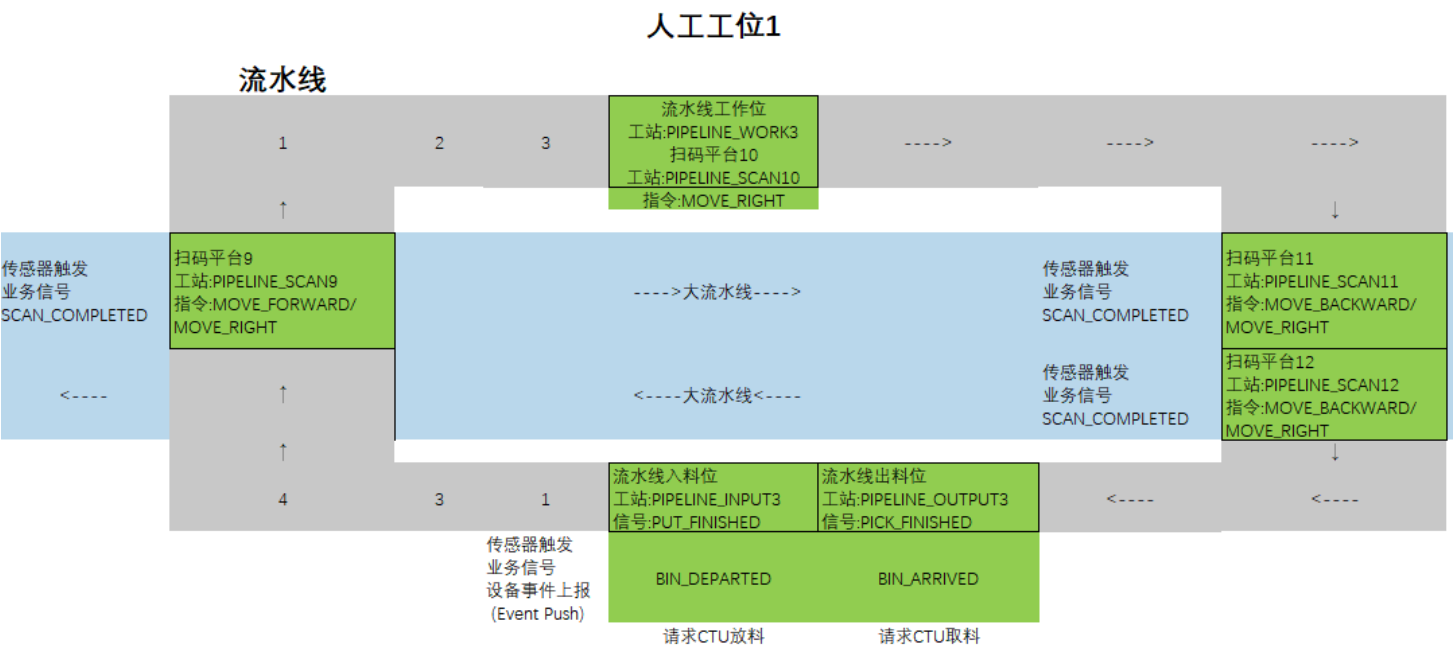
9.1 分拣机1布局图



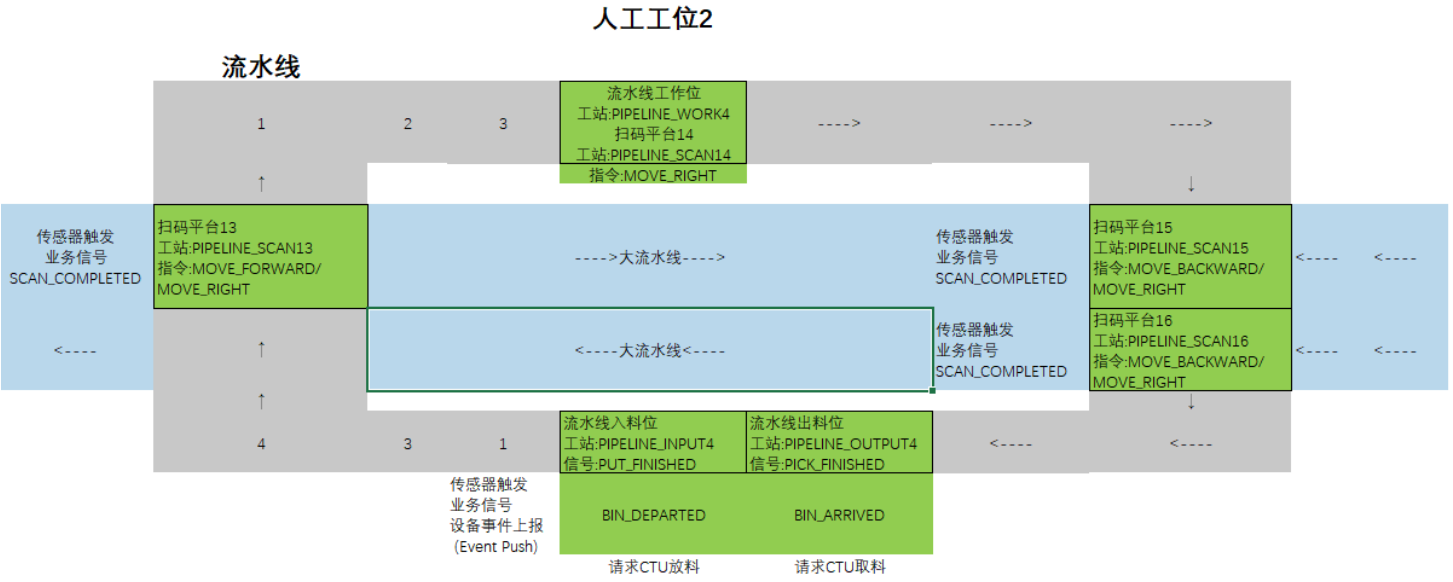
9.2 分拣机2布局图



9.3 人工工位1布局图

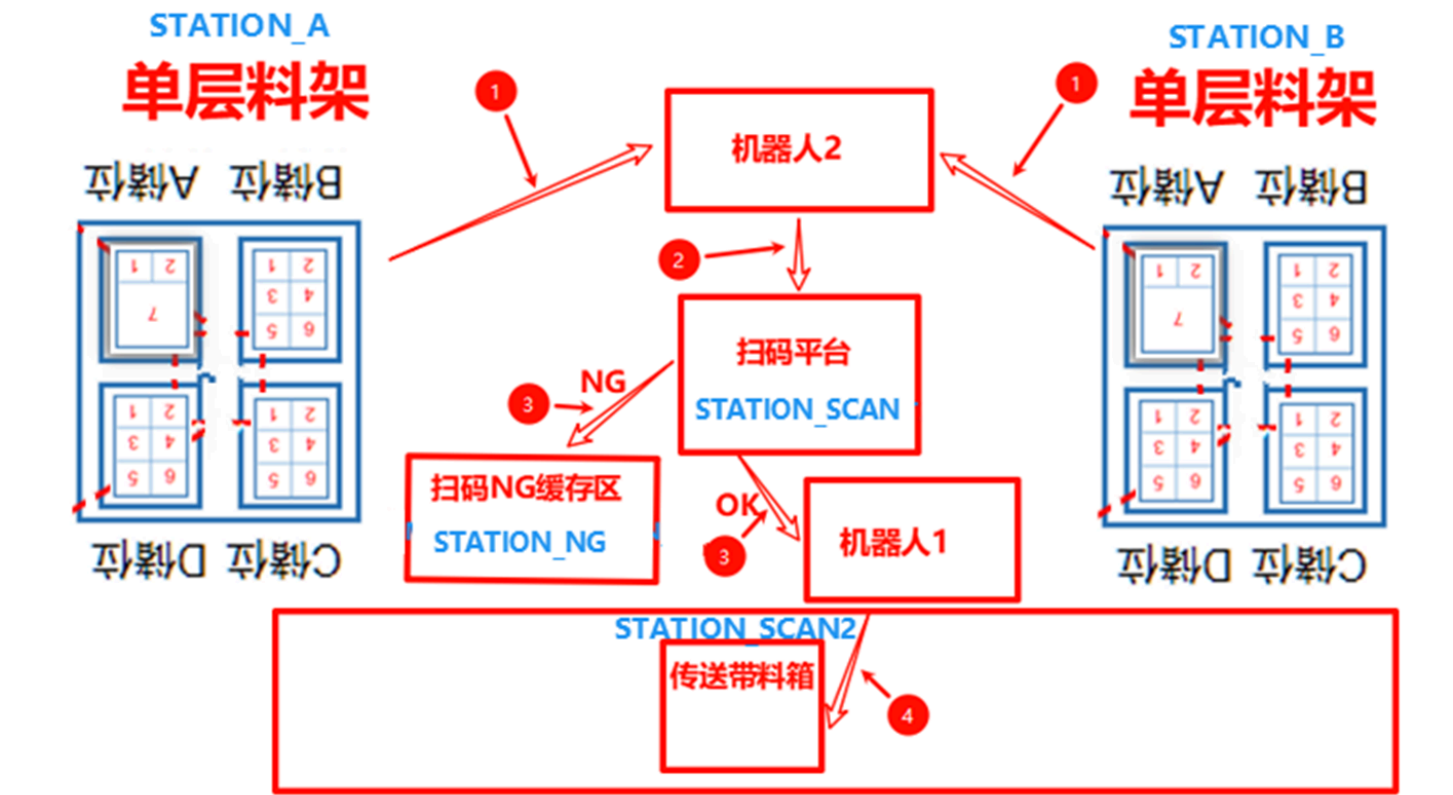


9.4 人工工位2布局图

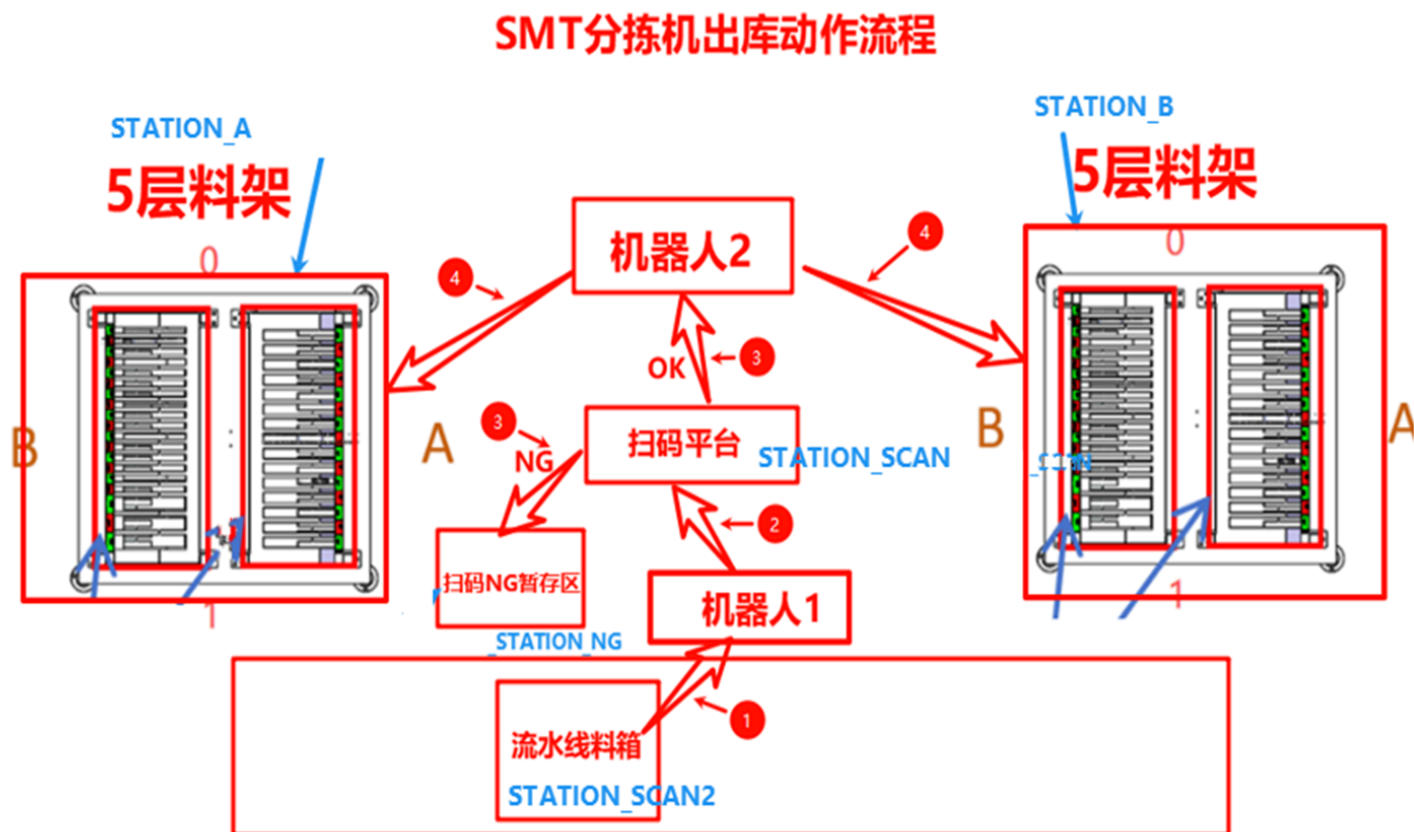


9.5 入库:单层料架储位布局 and 料箱单元格布局图及流程图

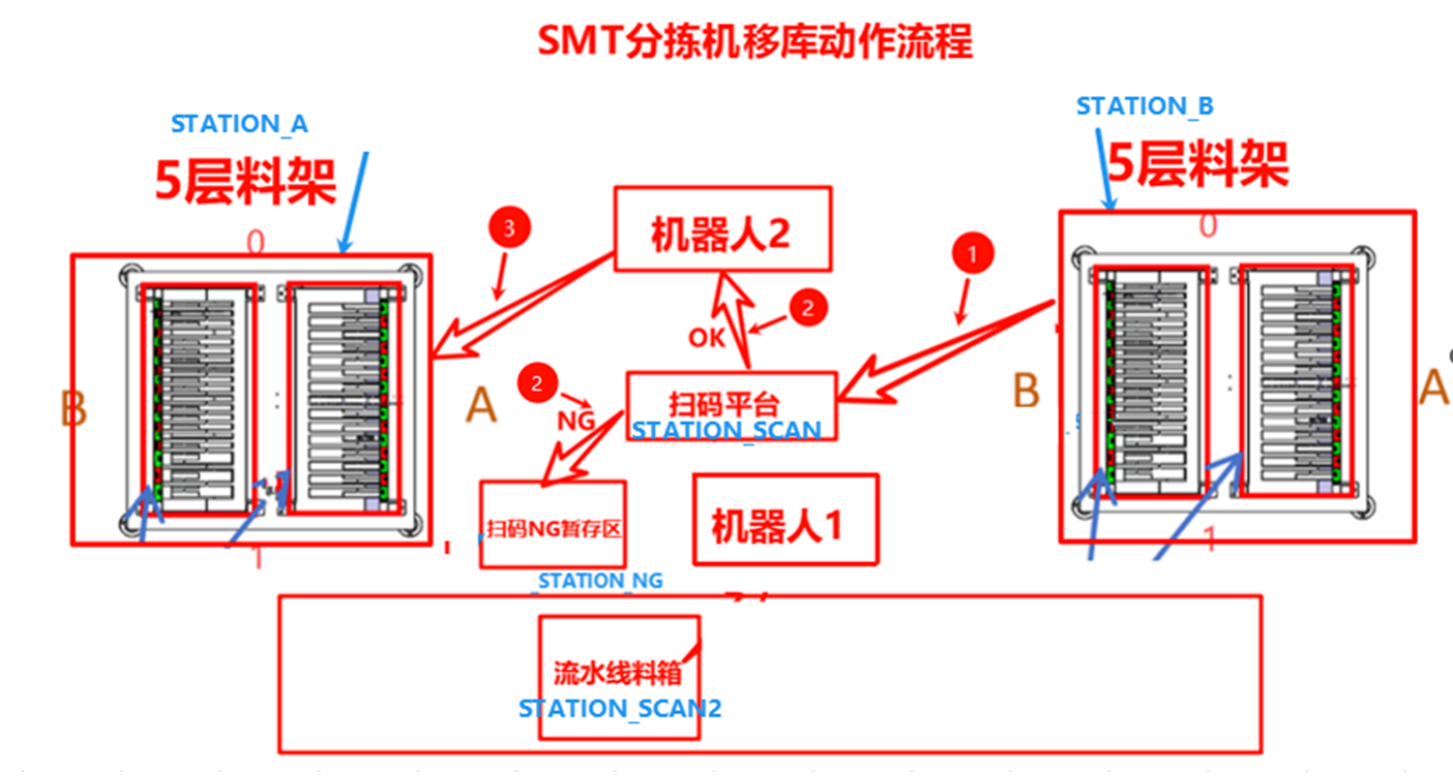
SMT分拣机入库动作流程



9.6 出库:五层料架储位布局和料箱单元格布局图及流程图



9.7 移库:五层料架储位布局图和料箱单元格布局图及流程图



10. 版本历史

版本	日期	说明
1.0.0	2026-03-18	初始版本

文档结束